

VM 3 - Remediation Tracker and Evidence Checklist

Supporting document for the penetration testing project

Target: 192.168.56.102 | Environment: Nagios XI + MySQL | Platform: CentOS / Docker

Purpose

This document summarizes the confirmed VM 3 findings, the evidence collected, the remediation actions applied or recommended, and the verification steps. It is designed to support the final penetration testing report with a clear remediation trail.

Quick Findings List

- Critical: Weak SSH credentials allowed root access.
- Critical: MySQL root password was exposed in Docker metadata.
- Critical: Exposed credentials allowed unauthorized MySQL access.
- Critical: Nagios XI configuration contained hardcoded database credentials.
- Critical: Sensitive user data and API keys were accessible from the database.
- High: Weak SHA1 htpasswd hashing was used for authentication files.
- High: MySQL port 3306 was exposed externally.
- High: Outdated Nagios Core / Nagios XI / MySQL / Apache components require upgrade.

Key message: the most serious attack path was weak root SSH access followed by credential discovery in Docker and Nagios XI configuration files. This could lead to database compromise, API key exposure, and full system control.

Detailed Finding Tracker

1. Weak SSH Credentials Allowed Root Access

Severity: Critical

Evidence:

- Root SSH access was obtained using a trivial password.
- Direct root login was enabled over SSH.
- The attacker could fully control the server after login.

Commands used:

```
ssh root@192.168.56.102
```

Impact: An attacker with root SSH access can read sensitive files, modify services, inspect Docker containers, retrieve credentials, and fully compromise the VM.

Remediation: The root password was changed, direct root SSH login was disabled, and SSH access should be restricted to trusted administrators.

Remediation commands:

```
passwd root
nano /etc/ssh/sshd_config
PermitRootLogin no
systemctl restart sshd
```

Verification:

```
grep -i PermitRootLogin /etc/ssh/sshd_config
systemctl status sshd
```

2. MySQL Root Password Exposed in Docker Metadata

Severity: Critical

Evidence:

- Docker metadata exposed the MySQL root password in plaintext environment variables.
- The value MYSQL_ROOT_PASSWORD=root was visible through Docker inspection.
- Anyone with Docker or root access could read the database password.

Commands used:

```
docker inspect dev_db | grep -i password
```

Impact: Exposed Docker environment variables can reveal credentials that attackers may use to access the database and retrieve or modify sensitive information.

Remediation: Rotate the exposed password and remove plaintext secrets from Docker environment variables. Use Docker secrets or a secure secrets manager where possible.

Remediation commands:

```
docker exec -it dev_db mysql -u root -p
ALTER USER 'root'@'localhost' IDENTIFIED BY 'NewStrongPassword@2026';
FLUSH PRIVILEGES;
```

Verification:

```
docker inspect dev_db | grep -i password
docker exec -it dev_db mysql -u root -p
```

3. Unauthorized MySQL Access Using Exposed Credentials

Severity: Critical

Evidence:

- The exposed MySQL root credentials allowed access to the database container.
- Database access was possible using credentials retrieved from Docker metadata.

Commands used:

```
docker exec -it dev_db mysql -u root -p
```

Impact: Database access can allow attackers to view, modify, or delete application data and create additional users or backdoors.

Remediation: Rotate database credentials, use least-privilege database accounts, and restrict MySQL access to required services only.

Remediation commands:

```
CREATE USER 'nagios_app'@'localhost' IDENTIFIED BY 'StrongPassword@2026';
GRANT SELECT, INSERT, UPDATE, DELETE ON nagiosxi.* TO 'nagios_app'@'localhost';
FLUSH PRIVILEGES;
```

Verification:

```
SELECT user, host FROM mysql.user;
SHOW GRANTS FOR nagios_app@localhost;
```

4. Hardcoded Nagios XI Database Credentials

Severity: Critical

Evidence:

- The Nagios XI configuration file contained a hardcoded database password in plaintext.
- The password value was readable from the configuration file inside the container.

Commands used:

```
docker exec -it nostalgic_ganguly /bin/bash
cat /usr/local/nagiosxi/html/config.inc.php | grep pwd
```

Impact: Hardcoded credentials can be read and reused by attackers to access the database or related application services.

Remediation: Remove plaintext credentials from configuration files and load secrets from protected environment variables or a secrets manager.

Remediation commands:

```
nano /usr/local/nagiosxi/html/config.inc.php
"pwd" => getenv("DB_PASSWORD")
export DB_PASSWORD='StrongPassword123!'
```

Verification:

```
grep -i "pwd" /usr/local/nagiosxi/html/config.inc.php
printenv DB_PASSWORD
```

5. Sensitive Data and API Keys Accessible from Database

Severity: Critical

Evidence:

- Using exposed database credentials, sensitive user information and API keys were queried from the xi_users table.
- API keys can allow unauthorized application access if reused.

Commands used:

```
mysql -u nagiosxi -p'n@gweb' nagiosxi
select username, api_key from xi_users;
```

Impact: Exposed API keys can allow attackers to impersonate users, access Nagios XI functions, or continue attacks against the application.

Remediation: Rotate exposed API keys, revoke unused keys, and review application logs for unauthorized API usage. If the Nagios XI database is not present in the current MySQL instance, verify the correct database location before rotation.

Remediation commands:

```
SHOW DATABASES;
Use the Nagios XI interface or correct application database to rotate exposed API keys.
```

Verification:

```
Confirm old API keys no longer work.
Review Nagios XI user/API settings.
```

6. Weak SHA1 htpasswd Hashing

Severity: High

Evidence:

- The htpasswd authentication file used SHA1-based {SHA} hashes.
- SHA1 is weak and vulnerable to offline cracking if the file is exposed.

Commands used:

```
find /etc /var /usr -type f -name "htpasswd" 2>/dev/null -exec grep -H "{SHA}" {} \;
```

Impact: Attackers who obtain the htpasswd file may crack weak SHA1 hashes and recover plaintext passwords.

Remediation: Replace SHA1 htpasswd hashes with stronger salted hashes and rotate affected passwords.

Remediation commands:

```
htpasswd -m /usr/local/nagiosxi/etc/htpasswd.users nagiosxi
htpasswd -m /usr/local/nagiosxi/etc/htpasswd.users nagiosadmin
```

Verification:

```
grep -H "{SHA}" /usr/local/nagiosxi/etc/htpasswd.users
cat /usr/local/nagiosxi/etc/htpasswd.users
```

7. MySQL Port 3306 Exposed Externally

Severity: High

Evidence:

- MySQL port 3306 was reachable from outside the database service.
- Exposed database ports increase attack surface and allow credential attacks or direct database access attempts.

Commands used:

```
iptables -L -n
nmap -p 3306 192.168.56.102
```

Impact: External MySQL exposure can allow unauthorized connection attempts, brute force, and direct attacks against the database service.

Remediation: Block external access to port 3306 and allow database connections only from trusted local services.

Remediation commands:

```
iptables -A INPUT -p tcp --dport 3306 -j DROP
iptables-save > /etc/sysconfig/iptables
```

Verification:

```
iptables -L -n
cat /etc/sysconfig/iptables
nmap -p 3306 192.168.56.102
```

8. Outdated Nagios Core / Nagios XI / MySQL / Apache Components

Severity: High

Evidence:

- The system used outdated components including Apache, MySQL, and Nagios Core/Nagios XI.
- Older versions may contain known vulnerabilities and may no longer receive security fixes.

Commands used:

```
nmap -sV -sC -p- 192.168.56.102
docker exec -it e1ec5de2d636 /usr/local/nagios/bin/nagios -v /usr/local/nagios/etc/nagios.cfg
```

Impact: Known vulnerabilities in outdated services can lead to remote code execution, privilege escalation, authentication bypass, or service compromise.

Remediation: Upgrade Nagios Core, Nagios XI, MySQL, Apache, OpenSSH, and container base images to supported versions. Apply security patches regularly.

Remediation commands:

```
Upgrade Nagios Core/Nagios XI to latest stable release.
Upgrade MySQL to MySQL 8.0 or a supported equivalent.
Patch Apache/OpenSSH and rebuild containers with supported base images.
```

Verification:

```
nmap -sV -sC -p- 192.168.56.102
Check vendor version output after upgrade.
```

Final Verification Checklist

- Root SSH password changed and direct root SSH login disabled.
- Docker metadata no longer exposes active database root credentials.
- MySQL root password rotated and application users follow least privilege.
- Nagios XI hardcoded credentials removed or replaced with secure secret loading.
- Exposed API keys rotated or revoked.
- htpasswd users migrated away from SHA1 hashes.
- Port 3306 blocked from external access and firewall rules saved persistently.
- Outdated software components scheduled for upgrade or replaced with supported versions.
- Before-and-after screenshots are included in the main report.

Recommended Submission Package

- Final VM 3 penetration testing report PDF.
- This VM 3 remediation tracker and evidence checklist.

- Screenshots showing each finding before remediation.
- Screenshots showing verification after remediation.
- Optional executive summary for a non-technical reviewer.

Final Notes

The strongest VM 3 findings are weak root SSH access and plaintext database credentials exposed through Docker/Nagios XI configuration. Together, these create a direct path to full system and database compromise. The remediations focus on credential rotation, access restriction, removal of plaintext secrets, stronger hashing, firewall hardening, and version upgrades.